VecCity: A Taxonomy-guided Library for Map Entity Representation Learning [Experiment, Analysis & Benchmark]

Wentao Zhang[1], Jingyuan Wang[1], Yifan Yang[1], Leong Hou U[2]
1. School of Computer Science and Engineering, Beihang University, Beijing, China
2. Department of Computer and Information Science, University of Macau, Macau SAR, China

arXiv:2411.00874v2 [cs.LG] 7 May 2025

ABSTRACT

Electronic maps consist of diverse entities, such as points of interest (POIs), road segments, and land parcels, playing a vital role in applications like ITS and LBS. Map entity representation learning (MapRL) generates versatile and reusable data representations, providing essential tools for efficiently managing and utilizing map entity data. Despite the progress in MapRL, two key challenges constrain further development. First, existing research is fragmented, with models classified by the type of map entity, limiting the reusability of techniques across different tasks. Second, the lack of unified benchmarks makes systematic evaluation and comparison of models difficult. To address these challenges, we propose a novel taxonomy for MapRL that organizes models based on functional modules—such as encoders, pre-training tasks, and downstream tasks—rather than by entity type. Building on this taxonomy, we present a taxonomy-driven library, *VecCity*, which offers easy-to-use interfaces for encoding, pre-training, fine-tuning, and evaluation. The library integrates datasets from nine cities and reproduces 21 mainstream MapRL models, establishing the first standardized benchmarks for the field. VecCity also allows users to modify and extend models through modular components, facilitating seamless experimentation. Our comprehensive experiments cover multiple types of map entities and evaluate 21 *VecCity* pre-built models across various downstream tasks. Experimental results demonstrate the effectiveness of *VecCity* in streamlining model development and provide insights into the impact of various components on performance. By promoting modular design and reusability, *VecCity* offers a unified framework to advance research and innovation in MapRL. The code is available at https://github.com/Bigscity-VecCity/VecCity.

1 INTRODUCTION

In the era of mobile internet, *electronic maps* have become a foundational platform for a wide range of applications, including intelligent transportation systems and location-based services. An electronic map consists of *map entities*, such as points of interest (POIs), road segments, and land parcels. These entities encapsulate complex geospatial data, spatial relationships, and geometric topological structures, presenting significant challenges for data representation. The effective representation of map entities has become a critical and enduring research focus within spatiotemporal data analysis and geographic information systems (GIS).

Traditionally, map entities have been stored as structured records in relational databases or geographic file formats, such as PostGIS [59], Shapefiles [18], and GeoJSON [4]. With this representation approach, services relying on map entities are required to develop task-specific models or algorithms to process the entities' attributes and extract geographic relationships within or between them. However, these task-specific models lack generalizability, limiting their reusability across different applications and scenarios.

In recent years, pre-trained representation learning has emerged as a powerful approach for generating versatile, task-independent data representations. It has achieved remarkable success across various domains, including natural language processing (NLP) [47, 48, 52] and computer vision (CV) [8, 16, 24]. The field of spatiotemporal data analysis has also embraced pre-trained representation learning to construct generalized representations of map entities [10, 34, 41, 45, 82], leading to the rise of an emerging research area known as Map Entity Representation Learning (MapRL). However, despite rapid advancements, two major structural challenges require further exploration to unlock the potential of MapRL.

Challenge 1: Fragmented Research Fields. Electronic maps consist of various types of map entities, such as POIs (points), road segments (polylines), and land parcels (polygons). Existing research often treats the modeling of these entities as separate areas, limiting the reusability of techniques, methods, and modules across fields. However, real-world electronic map applications typically require the integrated use of all three entity types. This segregation not only creates obstacles for practical applications but also forms implicit academic exchange barriers between subfields, thereby constraining the progress of MapRL research.

Challenge 2: Lack of Standardized Benchmark. Despite numerous models having been proposed, their performance is often evaluated on different datasets and under varying experimental settings. Unlike the CV and NLP fields, this domain lacks standardized datasets and benchmarks. This absence hinders fair comparisons between models, making it difficult to derive generalizable design principles.

To address the first challenge, we propose a novel taxonomy for MapRL models, namely method-based taxonomy, which organizes the essential components of an MapRL model into four key elements: *Map Data*, *Encoder Models*, *Pre-training Tasks*, and *Downstream Tasks*. Unlike traditional taxonomies that categorize models by map entity types, method-based taxonomy takes a functional approach, grouping encoder models into three paradigms—*Token-based*, *Graph-based*, and *Sequence-based*—each supporting specialized pre-training tasks. Only downstream tasks depend on the nature of the map entity. A key insight of our taxonomy is that the two core elements—Encoder Models and Pre-training Tasks—are not tightly coupled with specific map entity types. This decoupling of core components from entity-specific constraints fosters a more unified and transferable framework for MapRL, facilitating cross-domain integration and knowledge sharing.

To tackle the second challenge, we introduce the *VecCity* library, an MapRL model development toolkit guided by our taxonomy. The library organizes map entities and auxiliary data into three

atomic file types. Using these atomic files, we format data from nine cities into a unified structure. VecCity further standardizes model implementation with four interface functions covering encoders, pre-training tasks, and downstream tasks. Using these interfaces, we reproduce 21 mainstream MapRL models and conduct evaluations across city datasets, providing key insights for future research.

The main contributions of this paper are as follows:

• *Novel Taxonomy.* We present a novel taxonomy of MapRL models that goes beyond the traditional classification based on map entities, offering a unified classification system applicable to MapRL models across various types of map entities.

• *Easy-to-use Toolkit Library.* Guided by our taxonomy, we propose a toolkit, *VecCity*, which unifies the development process of MapRL models. To the best of our knowledge, *VecCity* is the first unified model toolkit specifically designed for MapRL.

• *Standard Benchmark. VecCity* implements 21 mainstream MapRL models and evaluates them on datasets from nine cities, establishing the first unified benchmark for MapRL models and offering valuable insights to guide future research.

2 TAXONOMY OF MAPRL

In this section, we present the proposed method-based taxonomy. Figure 1 (a) outlines the general framework for MapRL. The training process of a typical MapRL model consists of two key stages: *pre-training* and *fine-tuning*. During pre-training, the MapRL model takes map data as input, using an encoder model to transform them into representation vectors. The encoder's parameters are optimized through pre-training tasks. In the fine-tuning stage, downstream tasks are employed to further refine the encoder. This pipeline defines four core components of an MapRL model: *Map Data*, *Encoder Models*, *Pre-training Tasks*, and *Downstream Tasks*.

2.1 Map Data

The map data consists of map entities and auxiliary data.

2.1.1 Map Entities.

As illustrated in Fig. 1 (b), an electronic map consists of three geometric elements: *Points*, *Polylines*, and *Polygons*, each representing a distinct type of map entity.

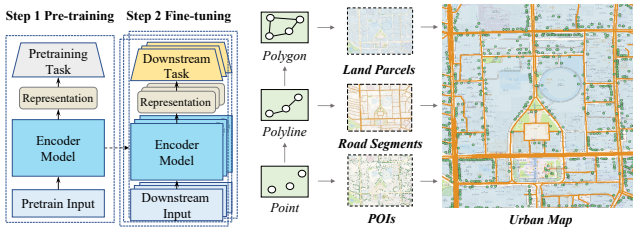
• *Points / POI.* A point element represents a *Point of Interest (POI)*, identified by a location coordinate, *i.e.,* $(latitude, longitude)$.

• *Polylines / Road Segment.* A polyline element represents a *Road Segment*, composed of a sequence of connected line segments, with each endpoint defined by location coordinates.

• *Polygons / Land Parcel.* A polygon represents a *Land Parcel*, delineated by a closed sequence of connected line segments.

For each map entity, a set of features describes its properties. For instance, the category feature of a POI differentiates the services it offers. Road segments are characterized by features like speed limits, lane counts, *etc.* Based on these definitions, we formally define electronic maps and map entities as follows.

Definition 1 (Electronic Maps and Map Entities)**.** An electronic map is defined as a set of map entities, $V = \{v_1, \ldots, v_i, \ldots, v_I\}$, where each entity v_i consists of four components: *ID*, *type*, *geometric shape*, and *feature*. The *ID* serves as the unique identifier of v_i. The *type* specifies the entity's geometric form – point, polyline, or



(a) The Framewrok of MapRL (b) Urban map with different map entities

Figure 1: Illustration of (a) the framework of a MapRL model and (b) an urban map with different map entities.

polygon. The *geometric shape* provides the coordinates that define its structure, while the *feature* captures its various attributes.

2.1.2 Auxiliary Data.

Besides map entities, two types of auxiliary data are commonly used in MapRL models: *Trajectories* and *Relation Networks*.

Definition 2 (Trajectories)**.** Trajectories refer to the mobility records of humans or vehicles. A trajectory is defined as a sequence of locations with corresponding timestamps, *i.e.,* $tr = (< l_{tr_1}, t_{tr_1} >, \ldots, < l_{tr_k}, t_{tr_k} >, \ldots, < l_{tr_K}, t_{tr_K} >)$, where l_{tr_k} and t_{tr_k} are the location and timestamp for the k-th trajectory sample.

Trajectory data can be divided into two classes based on the type of location samples l_{tr_k}. *i*) *Check-in Trajectories*: These use POIs as samples and are collected through social networks or LBS apps. They log a user's location only upon check-in, resulting in a low sampling rate, often requiring days to capture a single point. MapRL models leverage these trajectories to infer users' personal preferences. *ii*) *Coordinate Trajectories*: These rely on geographic coordinates as samples. Collected by ITS through GPS devices or mobile phones, they provide high-frequency sampling along with real-time speed and direction data. These trajectories are aligned with road segments using map matching algorithms [73] and facilitate route planning and dynamic state estimation for road segments.

Definition 3 (Relation Network)**.** A relation network represents map entities as a directed graph $G = \{V, \boldsymbol{E}\}$, where $V = \{v_1, \ldots, v_I\}$ represents the set of vertices corresponding to map entities. The adjacency matrix $\boldsymbol{E} \in \mathbb{R}^{I \times I}$ defines the relationships between entities, with each element e_{ij} indicating the relationship from v_i to v_j.

Network relations can be categorized into two types: *i*) *Geographical Relations (GR)*: These leverage spatial relationships between map entities to construct the adjacency matrix. Examples include connections between road segments in a road network [79] and distances between POIs [17, 85]. *ii*) *Social Relations (SR)*: These represent user behaviors, such as mobility trajectories, which are used to construct adjacency matrices. A typical example is *Origin-Destination (OD) flows*, widely used in MapRL models [81, 88], where e_{ij} in the adjacency matrix represents the number of trajectories that start from v_i and end at v_j.

2.2 Encoder Models and Pre-training Tasks

Encoder models and pre-training tasks are two core components of a pre-trained MapRL model, responsible for converting input map data into representation vectors for map entities. Our taxonomy

classifies encoder models into three types: *token-based*, *graph-based*, and *sequence-based*, designed to model features, relation networks, and trajectory sequences of map entities, respectively.

2.2.1 Token-based Models.

Encoders. The token-based encoder generates representation vectors directly from the features of map entities. For discrete features, it applies one-hot encoding, and for continuous features, the value range is divided into consecutive bins, each represented by a one-hot code. Given K features for a map entity, let the one-hot code of the k-th feature be $\boldsymbol{f}_k \in \{0,1\}^{F_k}$ (where F_k is the dimensionality). The encoder maps each feature to a learnable embedding space using a matrix $\boldsymbol{R}_k \in \mathbb{R}^{F_k \times d}$, generating the entity representation as $\boldsymbol{r} = \|_{k=1}^{K} \boldsymbol{f}_k^\top \cdot \boldsymbol{R}_k$, where $\|_{k=1}^{K}$ denotes the concatenation of embeddings from all features.

Existing token-based methods primarily innovate by refining how they extract meaningful features from map data, which can be classified into three key types: *i*) *Spatial Features*: These capture the locations and spatial structures of map entities. The most fundamental spatial feature is coordinate information. Hier [56] enhances spatial representation by embedding hierarchical spatial grids into rasterized maps. *ii*) *Temporal Features*: The characteristics of a map entity can vary across different times of the day. For instance, a restaurant's activity level differs between working hours and lunchtime. To model such variations, many token-based encoders divide the day into segments (e.g., hourly or half-hourly) and assign each segment a unique embedding to capture temporal dynamics [22, 36, 40, 61, 85]. *iii*) *Semantic Features*: These reflect the intrinsic properties of map entities, such as the functional types of POIs or the road types of segments. Some land parcel MapRL methods further enhance semantic features by counting the number of POI categories within a parcel [80].

Pre-training Tasks. For token-based encoders, the corresponding pre-training tasks include three types:

- *Token Relation Inference (TokRI).* This task aims to predict the relationship between two map entities based on their representation vectors. Specifically, given the representations $\boldsymbol{r}_i$ and $\boldsymbol{r}_j$ of two entities, the TokRI task predicts their relationship as $\hat{y}_{ij} = \phi(\boldsymbol{r}_i, \boldsymbol{r}_j)$, where $\phi(\cdot)$ is the prediction function, and $\hat{y}_{ij} \in [0,1]$ is the predicted probability of a relation existing. The encoder parameters are optimized during pre-training using cross-entropy loss.
- *Token Relation Contrastive Learning (TRCL).* This task ensures that entities with closer relationships are more likely to have similar representations. Given a representation sample $\boldsymbol{c}_i$, another sample $\boldsymbol{c}_j$ is treated as a positive sample if the corresponding entities v_i and v_j share a close relation; otherwise, $\boldsymbol{c}_j$ is treated as a negative sample. The encoder parameters are optimized using a contrastive learning loss, such as the InfoNCE loss [46].
- *Augmented Token Contrastive Learning (AToCL).* These tasks use data augmentation techniques to generate different views of a given entity's representation. A contrastive learning loss is then applied to maximize the similarity between views of the same entity while minimizing the similarity between different samples.

For TokRI and TRCL tasks, the primary innovation in existing models lies in how they construct relations. These methods can be categorized into three classes: *i*) *Distance-Based Relations*: This approach establishes relations based on spatial proximity. In TokRI tasks, RN2Vec [65] defines a relationship between entities if their geographical distance falls below a predefined threshold. In TRCL tasks, Teaser [85] selects the K nearest entities as positive samples for contrastive learning. *ii*) *Semantic-Based Relations*: Senmant [25], Place2V [72], and MT-POI [75] select entities that have the same or similar semantic features as positive samples for contrastive learning. *iii*) *Auxiliary Data-Based Relations*: This method leverages additional data, such as trajectories, to define relations. In TokRI tasks, P2Vec [21] and Tale [61] treat a trajectory as a sentence and the entities within it as words, using the Continuous Bag of Words (CBOW) model [44] to learn representations for "words" (map entities). In TRCL tasks, CatEM [3], R2Vec [27], and Teaser [85] employ the SkipGram model [44] with negative sampling to capture contextual relations along trajectories. When trajectory data is unavailable, "virtual trajectories" can be generated via random walks on relation networks [63]. Beyond trajectories, ZEMob [76] uses OD flows between entities to identify relationships.

For AToCL tasks, the innovation primarily lies in the methods used to generate augmented samples. For instance, ReMVC [80] and ReDCL [32] enhance contrastive learning by replacing or dropping specific features of the anchor entity to create augmented variants.

2.2.2 Graph-based Models.

Encoders. Graph-based encoders leverage auxiliary data in the form of relation networks (see Definition 3) to capture relationships among map entities. Graph-based encoders typically use the representations generated by token-based encoders as inputs and apply a graph neural network (GNN) to produce refined representation vectors. Specifically, given the map entity set $V = \{v_1, \ldots, v_I\}$, the embeddings generated by the token-based encoder are denoted as $\boldsymbol{R} = \{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_I\}$. The graph-based encoder then takes $\boldsymbol{R}$ along with the relation network G to generate the final representations: $\boldsymbol{H} = \{\boldsymbol{h}_1, \ldots, \boldsymbol{h}_i, \ldots, \boldsymbol{h}_I\} = \mathrm{GNN}(\boldsymbol{R}, G)$, where $\boldsymbol{h}_i$ is the representation vector for the i-th map entity, and $\mathrm{GNN}(\cdot)$ is the GNN-based encoder model. Compared to token-based models, graph-based encoders incorporate the relation networks into the entity representations, enriching their expressiveness.

Existing research on graph-based encoders focuses on developing GNNs capable of capturing complex relationships among map entities. Early studies applied standard GNN models, such as graph convolutional networks [71, 77] and graph attention networks [57, 78], to explore graphical [6, 28, 29] and social relations [38, 81]. Given the diverse relationships among map entities, some studies adopt more compound GNNs to enhance graph-based encoders. For example, the HREP model constructs a heterogeneous graph with four types of relations to represent land parcels [88]. The MGFN model utilizes human mobility patterns to build a multi-graph [70]. HRNR adopts a hierarchical GNN to capture the connection, structure, and function relations of road segments [69]. Recent advancements also investigate relationships across different types of map entities. For example, HGI [26] and HRoad [79] utilize hypergraphs to capture these inter-entity relationships.

Pre-training Tasks. For graph-based models, the corresponding pre-training tasks include four types:

- *Node Feature Inference (NFI).* This task aims to predict the features of map entities based on the representations generated

by graph-based encoders. Given an entity representation $\boldsymbol{h}_i$, the NFI task predicts its original features as $\hat{\boldsymbol{f}}_i = \phi(\boldsymbol{h}_i)$, where $\phi(\cdot)$ is typically an MLP-based prediction function. The objective is to minimize the prediction error.

• *Graph Autoencoder (GAu).* This task reconstructs the adjacency matrix of relation networks using an autoencoder framework. Given entity representations $\boldsymbol{H}$, the adjacency matrix is reconstructed as $\hat{\boldsymbol{E}} = \phi(\boldsymbol{H}\boldsymbol{H}^\top)$, where $\phi(\cdot)$ acts as a neural decoder. The graph encoder serves as the encoder of the auto-encoder, and the model is optimized to minimize the reconstruction error between $\hat{\boldsymbol{E}}$ and the original adjacency matrix $\boldsymbol{E}$.

• *Neighborhood Contrastive Learning (NCL).* The intuition behind this task is that connected map entities in a relation network should exhibit similar representations. This task treats connected entities as positive samples and unconnected entities as negative samples. Contrastive learning loss functions are then employed to optimize the encoder parameters.

• *Augmented Graph Contrastive Learning (AGCL).* This task introduces data augmentation to contrastive learning in relation networks. Given a relation network G and its augmented variant $\tilde{G}$, the representations of the same node across these versions are treated as positive samples, while different nodes serve as negative samples.

In NFI tasks, most methods leverage raw entity features as supervision signals [7, 69, 79]. In particular, GMEL [38] converts the OD flow relation network as labels for prediction. In the GAu task, the design of this task is closely linked to the structure of the GNN encoder. For example, HRNR [69] uses a three-layer hierarchical GNN, requiring different adjacency matrix reconstruction tasks per layer. HRoad [79] adopts a hypergraph in its encoder, so its pre-training step incorporates a hyperedge reconstruction task. In NCL tasks, the primary differences between models lie in the relation networks they utilize. For instance, HREP [88] and ReDCL [32] connect the nearest entities to incorporate spatial information. In AGCL tasks, the choice of data augmentation is critical. Randomly deleting edges can disrupt essential connections. To mitigate this, some methods use edge weights to determine deletion probabilities [6, 87].

2.2.3 Sequence-based Models.

Encoders. Sequence-based encoders are designed to capture temporal dependencies among map entities within trajectory-type auxiliary data. They employ sequential deep learning models, such as LSTMs [54], GRUs [13], or Transformers [60], to encode the embeddings of map entities along a trajectory into a sequence of representation vectors. These input embeddings are typically derived from token-based or graph-based encoders. Specifically, for a trajectory tr consisting of K entities, let their embedding sequence be $\boldsymbol{R}_{tr} = (\boldsymbol{r}_{tr_1}, \ldots, \boldsymbol{r}_{tr_k}, \ldots, \boldsymbol{r}_{tr_K})$. The sequence-based model then processes this input to generate corresponding representation vectors as $\boldsymbol{S}_{tr} = (\boldsymbol{s}_{tr_1}, \ldots, \boldsymbol{s}_{tr_k}, \ldots, \boldsymbol{s}_{tr_K}) = \mathrm{SEQ}(\boldsymbol{R}_{tr})$, where $\mathrm{SEQ}(\cdot)$ denotes the sequence-based encoder, and $\boldsymbol{s}_{tr_k}$ represents the embedding of the k-th entity along the trajectory tr.

In trajectory data, each location sample is associated with a timestamp. Integrating temporal information from timestamps into entity representations has become a key focus of sequence-based encoders. Early methods divide time into discrete slots, such as the hour of the day or the day of the week, and encoding these slots as embeddings using token-based encoders [21, 42, 85]. More advanced models embed timestamps within the components of sequential models. For example, JCLRNT [42], Toast [12], and CTLE [35] incorporate timestamps into the positional encoding of Transformers. Similarly, START [28] integrates travel time into the attention mechanism, enhancing the model's ability to capture temporal dependencies.

Pre-training Tasks. For sequence-based models, the corresponding pre-training tasks include three types:

• *Trajectory Prediction (TrajP).* The goal of this task is to predict the second half of a trajectory using the first half. Given a trajectory tr, this task is formally defined as $\hat{tr}_{[k+1:K]} = Pre(\boldsymbol{S}_{[1:k]})$, $\boldsymbol{S}_{[1:k]} = Enc(tr_{[1:k]})$, where $Enc(\cdot)$ is the sequence-based encoder, and $Pre(\cdot)$ is the prediction function. Here, $tr_{[1:k]}$ represents the first k samples of the trajectory, and $s_{[1:k]}$ is their corresponding sequence of representation vectors. The objective is to minimize the distance between $\hat{tr}_{[k+1:K]}$ and the ground truth.

•*Masked Trajectory Recovery (MTR).* Inspired by the masked language model (MLM) in NLP [15, 37], this task randomly masks parts of a trajectory and trains an autoencoder for reconstruction. Given a trajectory tr and its masked version $\tilde{tr}$, the recovery process follows $\hat{tr} = Dec(\tilde{\boldsymbol{S}})$, $\tilde{\boldsymbol{S}} = Enc(\tilde{tr})$, where $Enc(\cdot)$ is the sequence-based encoder, and $Dec(\cdot)$ is the decoder. The objective is to minimize the reconstruction error between tr and $\hat{tr}$.

• *Augmented Trajectory Contrastive Learning (ATrCL).* This task generates multiple versions of a trajectory using data augmentation techniques [19, 84]. Sequence-based encoders then encode both the original and augmented trajectories, and a contrastive learning loss is applied to enhance the similarity between representations of the same trajectory while distinguishing them from others.

In TrajP tasks, the prediction labels can be various. For example, Hier [56] uses entity IDs as prediction labels, while CTLE [35] employs visiting time as labels to incorporate temporal information. In MTR tasks, different strategies exist for selecting masked positions. A basic approach is to mask positions within trajectories [35] randomly, but this overlooks the sequential dependencies between samples. To address this, START [28] and Toast [12] mask continuous subsequences within a trajectory. In ATrCL tasks, models adopt diverse augmentation strategies. JCLRNT [42] generates augmented trajectories by randomly deleting or replacing trajectory points. Considering contextual dependencies, START [28] replaces continuous sub-sequences, while CACSR [22] injects white noise into representations to enhance robustness.

2.2.4 Module Integration of Encoders and Pre-training Tasks.

The encoders and pre-training tasks introduced in the previous sections serve as the foundational components of MapRL models. A complete MapRL model can integrate multiple types of encoders and pre-training tasks, leveraging their complementary strengths to enhance representation learning. For the encoder modules, existing models typically follow a "token → graph → sequence" modeling pipeline. This pipeline first extracts basic features using token-based encoders, then captures relational structures through graph-based encoders, and finally models temporal dependencies with sequence-based encoders. At each stage of this pipeline, a MapRL model may employ multiple encoders simultaneously for different pre-training tasks. Table 1 provides an overview of how mainstream models combine encoders and pre-training tasks.

Table 1: Summary of existing MapRL models. "*" indicates the reproduced models in the *VecCity* library.

	Method	Taxnomy Category	Encoders			Pretraining Tasks										Map Entity			Features			Aux. Data	
			Token	Graph	Seq	TokRI	TRCL	AToCL	NFI	GAu	NCL	AGCL	TrajP	MTR	ATrCL	POI	RS	LP	Sem	Spa	Temp	Traj	Rel
Point of Interest	CWAP [36]	Token-based	✓				✓									✓					✓	CIT	
	CAPE [5]	Token-based	✓				✓									✓			✓			CIT	
	MT-POI [75]	Token-based	✓				✓									✓			✓	✓			GR
	Place2V [72]	Token-based	✓				✓									✓			✓	✓			GR
	Semant [25]	Token-based	✓				✓									✓			✓	✓			GR
	CatEM [3]	Token-based	✓				✓									✓			✓	✓		CIT	
	SkipG* [44]	Token-based	✓				✓									✓						CIT	
	Tale* [61]	Token-based	✓			✓										✓					✓	CIT	
	Teaser* [85]	Token-based	✓				✓									✓				✓	✓	CIT	GR
	Hier* [56]	Seq.-based	✓		✓								✓			✓				✓		CIT	GR
	P2Vec* [21]	Token-based	✓			✓										✓				✓		CIT	GR
	CACSR* [22]	Seq.-based	✓		✓										✓	✓					✓	CIT	
	CTLE* [35]	Seq.-based	✓		✓								✓	✓		✓				✓	✓	CIT	
Road Segment	DyToast [11]	Graph-based	✓	✓			✓		✓					✓	✓		✓			✓		CDT	GR
	R2Vec* [27]	Token-based	✓				✓										✓			✓			GR
	TrajRNE [55]	Graph-based	✓	✓		✓				✓							✓		✓			CDT	GR/SR
	JGRM [40]	Seq.-based	✓	✓	✓									✓	✓		✓		✓	✓	✓	CDT	GR
	RN2Vec* [65]	Token-based	✓			✓											✓		✓	✓			GR
	HRNR* [69]	Graph-based	✓	✓						✓							✓		✓	✓			GR
	SARN* [6]	Graph-based	✓	✓								✓					✓		✓	✓			GR
	Toast* [12]	Seq.-based	✓		✓		✓							✓	✓		✓				✓	CDT	GR
	HRoad* [79]	Graph-based	✓	✓					✓	✓							✓		✓	✓			GR
	START* [28]	Seq.-based	✓	✓	✓									✓	✓		✓		✓	✓	✓	CDT	GR
	JCLRNT* [42]	Seq.-based	✓	✓	✓							✓			✓		✓		✓	✓	✓	CDT	GR
Land Parcel	CGAL [83]	Graph-based	✓	✓						✓						✓		✓	✓	✓			GR/SR
	DLCL [17]	Graph-based	✓	✓						✓						✓		✓	✓	✓			GR/SR
	HUGAT [31]	Graph-based	✓	✓						✓								✓	✓	✓	✓		GR/SR
	Re2Vec [39]	Graph-based	✓	✓						✓						✓		✓	✓	✓			GR/SR
	ReCP [33]	Token-based	✓			✓	✓	✓								✓		✓	✓	✓			SR
	HGI [26]	Graph-based	✓	✓			✓	✓								✓		✓	✓	✓			GR
	ReDCL [32]	Token-based	✓			✓	✓	✓								✓		✓	✓	✓			GR
	HAFus [58]	Graph-based	✓	✓						✓						✓		✓	✓	✓			GR/SR
	ZEMob* [76]	Token-based	✓				✓											✓			✓		SR
	GMEL* [38]	Graph-based	✓	✓					✓	✓								✓		✓			GR/SR
	MGFN* [70]	Graph-based	✓	✓						✓								✓		✓			SR
	HDGE* [63]	Token-based	✓				✓											✓		✓	✓		GR/SR
	MVURE* [81]	Graph-based	✓	✓						✓						✓		✓		✓			SR
	ReMVC* [80]	Token-based	✓					✓								✓		✓	✓	✓			SR
	HREP* [88]	Graph-based	✓	✓						✓	✓					✓		✓		✓			GR/SR

For the three-stage encoder pipeline, MapRL models follow two pre-training paradigms. *i*) *Sequential Training*: Encoders are trained in stages—first token-based, then graph-based, and finally sequence-based—where each stage refines the output of the previous one. *ii*) *Joint Training*: All encoders are optimized simultaneously using multi-task learning, enabling the model to capture entity features, structural relationships, and temporal patterns in an integrated manner. These paradigms offer flexibility in model impanelments, depending on the specific requirements of the MapRL task.

2.3 Downstream Tasks

The downstream tasks consist of two components: *Downstream Models* and *Fine-tuning Loss*. Downstream models use the representation vectors generated by the encoders to perform specific tasks. Fine-tuning loss adjusts the encoder parameters to improve the adaptability of the representation vectors for these tasks. In contrast to pre-training tasks, which are closely tied to the encoders, downstream tasks are heavily influenced by the type of map entities. This subsection presents a taxonomy of downstream tasks.

Tasks for POIs. Common downstream tasks for POI entities can be classified into three categories, all of which utilize cross-entropy loss as the objective function:

• *POI Classification (POIC)*: This task aims to classify a POI into predefined categories based on its representation. A typical downstream model consists of an MLP followed by a SoftMax classifier.

• *Next POI Prediction (NPP)*: This task converts a POI-based trajectory into a sequence of representation vectors using an encoder. The downstream model, often a deep sequential model like LSTM, predicts the next POI in the trajectory using the representation of the first half. The output is a multi-class classification, with each class representing a possible POI.

• *Trajectory User Link (TUL)*: This task predicts the user who generated a trajectory based on the trajectory's representation sequence. The downstream model is a deep sequential model, with each output class corresponding to a user ID.

Tasks for Road Segments. Common downstream tasks for road segment entities fall into three categories:

• *Average Speed Inference (ASI)*. This task leverages the representation vector to infer the average speed of a road segment. The average speed is derived from trajectory data and must be excluded from the input features of the representation encoders to prevent data leakage. Inferring average speed is a representative task for estimating unknown features of a road segment. The downstream model for this task is a linear regression function, and the fine-tuning loss function is Mean Square Error (MSE).

• *Travel Time Estimation (TTE)*. This task estimates the travel time from the origin to the destination along a trajectory composed of road segments. The downstream model is a deep sequential model. Its input is the representation sequence of the segment-based trajectory, and the output is the regressed travel time. The fine-tuning loss function is MSE.

• *Similarity Trajectory Search (STS)*. This task retrieves the most similar trajectory from a database given a query. A deep sequential model encodes all trajectories into representation vectors, and

cosine similarity identifies the closest match. Fine-tuning uses a contrastive loss (e.g., InfoNCE), treating the ground-truth trajectory—formed by replacing a sub-trajectory with one sharing the same endpoints [6, 28, 42]—as the positive sample.

Tasks for Land Parcels. The downstream tasks for land parcel entities primarily focus on attribute classification, such as function and population, and mobility volume. These tasks are typically categorized into three types:

• *Land Parcel Classification (LPC)*: This task predicts specific labels for a land parcel, such as land usage or functional category, based on its representation vector. The downstream model is an MLP with a SoftMax classifier, optimized with cross-entropy loss.

• *Flow Inference (FI)*: This task predicts the incoming and outgoing population of a land parcel over a given period (*e.g.*, one month) using its representation vector. The downstream function is usually a linear regression, with MSE as the loss.

• *Mobility Inference (MI)*: This task predicts the OD flow between pairs of land parcels. The input to the downstream function consists of the representations of two parcels, and the output is the predicted OD flow volume. The downstream function can be a neural network, with MSE as the fine-tuning loss.

There are two fine-tuning strategies for downstream tasks [6, 53]. *i*) *Downstream Fine-tuning*: In this strategy, only the downstream models are fine-tuned, while the parameters of the MapRL encoders remain frozen. *ii*) *End-to-End Fine-tuning*: This approach tunes the parameters of both the downstream models and the MapRL encoders, enabling the entire model to be optimized jointly.

3 THE LIBRARY – *VECCITY*

Based on the method-based taxonomy of map entity representation learning models, we propose *VecCity*, an easy-to-use toolkit for MapRL model development. Figure 2 illustrates the framework of the *VecCity* library, comprising three functional modules aligned with the four key elements of MapRL models outlined in Section 2. *i*) *Data Module*: Corresponding to the map data element in Section 2.1, this module handles data preparation and processing. *ii*) *Upstream Module*: Aligned with the encoder and pre-training elements in Section 2.2, it is responsible for representation encoding and model pre-training. *iii*) *Downstream Module*: Linked to the downstream task element in Section 2.3, this module manages model fine-tuning and performance evaluation on downstream tasks.

3.1 Data Module and Atomic Files

In the Data Module, map entities and auxiliary data are represented in a unified file format, called *Atomic Files* [64]. This format consists of three file types: *.geo* – represents map entities, *.traj* – stores trajectory data, *.rel* – captures relation networks. Using the three types of atomic files, we can represent map entities and auxiliary data in a unified format. Original data often comes in various spatiotemporal formats, such as GeoJSON [4], Shapefile [18], and NPZ [23], requiring multiple data-loading implementations and hindering reusability. Our *VecCity* library solves this issue by adopting unified atomic files, simplifying data processing.

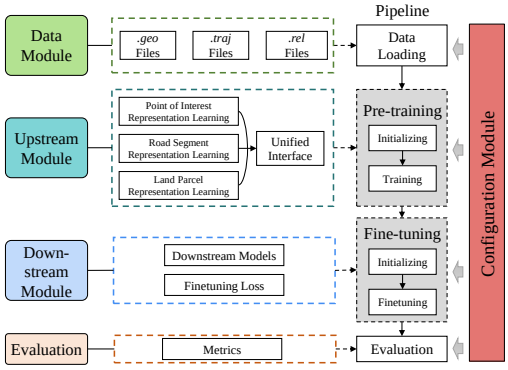


Figure 2: The overall pipeline for *VecCity*.

In *VecCity*, we have pre-loaded map entities and auxiliary data from nine cities: New York (NY), Chicago (CHI), Tokyo (TYO), Singapore (SIN), Porto (PRT), San Francisco (SF), Beijing (BJ), Chengdu (CD), and Xi'an (XA), using public data sources. Map entities are extracted from OpenStreetMap (OSM), an open-source global map. POI check-in trajectories come from Foursquare datasets [74], while coordinate trajectories are collected from various studies [1, 28, 43, 49]. The trajectory data preprocessing approach aligns with prior studies [20]. Relation networks are OD flow networks constructed from these trajectory datasets. All datasets are processed, converted into atomic files, and integrated into the *VecCity* library. Details of the datasets are provided in Tab. 2. Users can directly apply these datasets in their model development. Additionally, we provide data conversion scripts on the *VecCity* GitHub repository, enabling users to convert their private data into the atomic file format.

3.2 Upstream Module and Downstream Module

Upstream Module. Based on the taxonomy of MapRL encoders and pre-training tasks in Section 2.2, the *VecCity* library provides the upstream module through two key interfaces: *encode()* and *pretraining_loss()*.

• *encode()*: This interface implements the encoder for an MapRL model. It takes data extracted from atomic files as input and outputs representation vectors for map entities.

• *pretraining_loss()*: This interface manages the implementation of pre-training tasks. It prepares samples through methods such as data masking, generating positive/negative samples, or data augmentation, and calculates the corresponding loss values.

In the *VecCity* framework, users can execute the pre-training phase of an MapRL model by implementing the interface functions in the upstream module. Leveraging these interfaces, we have reproduced 21 mainstream MapRL models in *VecCity*, including 7 models each for POI, road segment, and land parcel representation learning. The reproduced models are marked with "*" in Tab. 1.

Downstream Module. The downstream module provides two key interfaces: *downstream_model()* and *finetuning_loss()*.

• *downstream_model()*: This interface implements the downstream models used during the fine-tuning step. It typically utilizes a simple MLP or LSTM model. For classification tasks, the output layer is a SoftMax classifier, while for continuous prediction tasks, it is a linear regression layer.

Table 2: Pre-prepared datasets for the *VecCity* library.

City	#POI	#Segment	#Parcel	Check-in Traj		Coor. Traj		#OD
				#Traj	#User	#Traj	#User	
NY	79,627	90,781	262	823,853	2,758	-	-	28,189
CHI	28,141	47,669	77	279,183	921	-	-	4,631
TYO	61,858	407,905	64	573,703	2,227	226,782	-	-
SIN	75,329	35,084	332	827,828	3,489	11,170	-	-
PRT	4,521	11,095	382	19,402	126	695,085	435	324
SF	15,674	27,274	194	171,873	361	500,516	405	24,716
BJ	81,181	40,306	11,208	-	-	1,018,312	1,677	687,890
CD	17,301	6,195	1,306	-	-	559,729	48,295	64,482
XA	19,108	5,269	1,056	-	-	384,618	26,787	54,365

• *finetuning_loss()*: This interface defines the loss function for fine-tuning. Cross-entropy is used for classification tasks, MSE for regression tasks, and InfoNCE for contrastive learning tasks.

Using these two interfaces, we have implemented all nine downstream tasks listed in Section 2.3. Users can directly invoke these pre-set tasks to fine-tune their models.

3.3 Evaluation & Config Modules

The evaluation module provides an *evaluation()* interface to assess MapRL model performance across downstream tasks. the *VecCity* library supports various metrics tailored to different task types: MAE, MSE, RMSE, R^2, and MAPE for regression tasks; Accuracy@k, Recall@k, F1@k, and Mean Rank (MR) for classification tasks; and Accuracy@k and MR for trajectory similarity search.

The config module serves as the control center, allowing users to configure the entire "pre-training → fine-tuning → evaluation" pipeline through a configuration file. To train and evaluate a model with *VecCity*, users simply specify the required pre-built or custom models and functions in the configuration file. The library then automatically generates the necessary script to run experiments. Users only need to execute the script, and *VecCity* will invoke the interface functions of each module to complete the experiment.

4 PERFORMANCE BENCHMARK

In this section, we present performance comparison experiments for the pre-built MapRL models in *VecCity*. These experiments serve three purposes: *i*) Demonstrating Usability and Power: To show that *VecCity* is a user-friendly and powerful toolkit for MapRL model development and evaluation. *ii*) Establishing Performance Benchmarks: To provide standard performance comparisons of mainstream MapRL models using the datasets pre-loaded in *VecCity*. *iii*) Analyzing Model Efficiency. Evaluating the computational efficiency of different MapRL models to offer insights for designing more efficient approaches.

4.1 Datasets and Experimental Setups

4.1.1 Datasets. The dataset selection for our experiments follows these guiding principles: for each type of MapRL model, we incorporate all datasets that meet the task requirements. Specifically, for POI-oriented MapRL models, we use datasets from New York, Chicago, Tokyo, Singapore, and San Francisco. For road-segment-oriented models, the datasets include Beijing, Chengdu, Xi'an, Porto, and San Francisco. For land-parcel-oriented models, we use datasets from Beijing, Chengdu, Xi'an, Porto, and San Francisco. Each dataset is divided into training, validation, and test sets following a 6:2:2 ratio, ensuring a balanced and reliable performance evaluation. For each downstream task, we apply end-to-end fine-tuning to achieve optimal performance.

Table 3: Performance comparison of POI MapRL methods across various datasets (Mean ± Std). Bold and underlined text indicates the best and second-best results, respectively. '↑' denotes that a higher value is better, while '↓' indicates that a lower value is preferred.

Data	Task	Metric	SkipG	Tale	Teaser	Hier	P2Vec	CACSR	CTLE
New York	POIC	ACC@1↑	0.07±0.00	0.06±0.01	0.07±0.01	0.07±0.01	**0.08±0.01**	0.06±0.02	0.06±0.01
		F1↑	0.01±0.00	0.02±0.01	0.01±0.01	0.01±0.00	**0.02±0.01**	0.01±0.00	0.01±0.00
	NPP	ACC@1↑	0.10±0.02	0.14±0.01	0.10±0.02	0.12±0.00	0.13±0.01	0.14±0.05	**0.15±0.01**
		ACC@5↑	0.25±0.04	0.33±0.00	0.25±0.05	0.29±0.01	0.16±0.00	0.33±0.04	**0.33±0.01**
	TUL	ACC@1↑	0.57±0.03	0.67±0.00	0.57±0.02	0.58±0.03	0.64±0.01	0.64±0.03	**0.69±0.00**
		F1↑	0.26±0.03	0.44±0.01	0.27±0.03	0.28±0.04	0.33±0.00	0.39±0.07	**0.45±0.01**
Tokyo	POIC	ACC@1↑	0.35±0.01	0.22±0.00	**0.38±0.02**	0.34±0.04	0.32±0.00	0.30±0.04	0.30±0.04
		F1↑	0.02±0.01	**0.05±0.01**	0.02±0.01	0.01±0.01	0.04±0.00	0.02±0.00	0.02±0.00
	NPP	ACC@1↑	0.16±0.00	0.16±0.01	0.16±0.01	**0.16±0.00**	0.16±0.01	0.16±0.06	0.16±0.00
		ACC@5↑	0.32±0.01	0.30±0.00	0.30±0.04	**0.34±0.00**	0.33±0.01	0.33±0.01	0.33±0.01
	TUL	ACC@1↑	0.43±0.01	0.40±0.01	0.44±0.02	0.44±0.03	0.45±0.00	0.46±0.10	**0.50±0.00**
		F1↑	0.23±0.01	0.23±0.01	0.23±0.01	0.23±0.01	0.25±0.00	0.28±0.10	**0.29±0.00**
Chicago	POIC	ACC@1↑	0.07±0.00	0.10±0.00	0.07±0.01	0.08±0.00	**0.12±0.01**	0.10±0.01	0.10±0.02
		F1↑	0.01±0.00	0.02±0.01	0.00±0.00	0.01±0.00	**0.02±0.01**	0.02±0.01	0.02±0.01
	NPP	ACC@1↑	0.08±0.02	0.10±0.01	0.06±0.03	0.10±0.03	0.15±0.01	0.15±0.05	**0.18±0.01**
		ACC@5↑	0.24±0.05	0.24±0.01	0.18±0.08	0.26±0.06	0.36±0.01	**0.40±0.04**	0.38±0.01
	TUL	ACC@1↑	0.46±0.08	0.62±0.01	0.43±0.07	0.55±0.02	0.66±0.01	0.67±0.07	**0.76±0.01**
		F1↑	0.20±0.07	0.37±0.01	0.19±0.01	0.30±0.01	0.42±0.01	0.44±0.06	**0.60±0.03**
Singapore	POIC	ACC@1↑	0.09±0.04	0.04±0.01	0.06±0.00	0.12±0.04	0.07±0.00	0.10±0.04	**0.13±0.03**
		F1↑	0.00±0.00	0.01±0.01	0.01±0.01	0.00±0.00	**0.03±0.01**	0.01±0.00	0.00±0.00
	NPP	ACC@1↑	0.03±0.00	0.05±0.01	0.03±0.00	0.05±0.02	0.08±0.01	**0.11±0.04**	0.10±0.03
		ACC@5↑	0.06±0.01	0.10±0.00	0.07±0.01	0.11±0.03	0.18±0.01	**0.18±0.05**	0.14±0.05
	TUL	ACC@1↑	0.26±0.06	0.31±0.01	0.27±0.06	0.27±0.08	0.40±0.01	0.40±0.05	**0.44±0.03**
		F1↑	0.11±0.04	0.17±0.01	0.14±0.01	0.16±0.01	0.18±0.01	0.18±0.07	**0.25±0.00**
San Francisco	POIC	ACC@1↑	0.06±0.03	0.01±0.00	0.06±0.01	**0.07±0.02**	0.04±0.01	0.06±0.02	0.06±0.01
		F1↑	0.00±0.00	0.01±0.01	0.00±0.00	0.00±0.00	**0.02±0.01**	0.01±0.01	0.00±0.00
	NPP	ACC@1↑	0.02±0.01	0.01±0.00	0.02±0.00	0.04±0.01	0.03±0.01	0.03±0.03	**0.06±0.00**
		ACC@5↑	0.06±0.01	0.06±0.01	0.06±0.01	0.10±0.01	0.06±0.01	0.06±0.06	**0.12±0.01**
	TUL	ACC@1↑	0.25±0.08	0.34±0.01	0.30±0.11	0.35±0.12	0.52±0.01	0.53±0.06	**0.61±0.01**
		F1↑	0.13±0.01	0.15±0.00	0.10±0.06	0.15±0.05	0.26±0.01	0.29±0.02	**0.32±0.01**
Per. Avg. Rank			5.8	5.7	4.5	4.2	3.0	2.9	**2.4**

4.1.2 Experimental Settings. In our experiments, the dimension of map entity representation vectors across all models is set to 128. For other model configurations, such as the number of layers and the dimensions of intermediate variables, we adopt the default settings from the original papers as initial values. We then use a trial-and-error approach to tune the parameters on each dataset, searching for the best performance configurations. Each model is run five times with different seeds, and the results are reported as the mean and standard deviation. All experiments are executed on an Ubuntu 20.04 system equipped with NVIDIA GeForce 3090 GPUs. The default batch size during model training is set to 64. In cases of out-of-memory (OOM) issues, the batch size is halved iteratively, with a minimum batch size of 8.

4.2 Overall Experimental Results

4.2.1 Experimental Results for POI MapRL. Table 3 shows the performance of MapRL models on three downstream tasks: POI Classification (POIC), Next POI Prediction (NPP), and Trajectory User Link (TUL), each evaluated using two types of metrics. The table also reports average performance rankings across tasks (Pre Ave Rank) and the ranking based on the models' average performance (Pre Rank). From Tab. 3, we have the following observations.

Table 4: Performance comparison of Road Segment MapRL methods on various datasets.

Data	Task	Metric	RN2Vec	HRNR	SARN	Toast	HRoad	START	JCLRNT
Beijing	ASI	MAE↓	3.58±0.01	3.16±0.00	2.81±0.00	2.74±0.01	**2.71±0.01**	2.72±0.01	2.80±0.02
		RMSE↓	5.73±0.00	5.59±0.00	5.48±0.00	5.51±0.01	5.41±0.00	**5.39±0.00**	5.48±0.01
	TTE	MAE↓	661±14.0	631±19.3	596±20.8	533±33.0	585±16.3	**512±19.9**	514±38.3
		RMSE↓	2978±48	2968±42	2581±84	2576±79	2554±35	2502±37	**2464±36**
	STS	ACC@3↑	0.76±0.03	0.82±0.03	0.84±0.03	0.79±0.10	0.87±0.01	**0.90±0.03**	0.90±0.04
		MR↓	12.4±1.54	11.5±1.94	13.30±2.17	12.8±2.81	14.0±1.42	10.9±1.98	**10.4±1.15**
Chendu	ASI	MAE↓	6.57±0.06	6.42±0.04	5.99±0.02	6.36±0.03	**5.93±0.02**	6.05±0.01	6.35±0.08
		RMSE↓	14.0±0.07	13.7±0.07	13.3±0.04	13.3±0.16	**13.2±0.00**	13.5±0.01	13.5±0.13
	TTE	MAE↓	106±0.06	100±0.04	**88.3±4.27**	95.0±2.44	88.5±5.48	95.8±3.12	97.0±4.08
		RMSE↓	155±0.07	150±0.05	157±6.70	150±2.75	**137±5.67**	151±3.84	148±3.79
	STS	ACC@3↑	0.69±0.04	0.73±0.06	0.68±0.13	0.69±0.02	0.68±0.05	**0.76±0.03**	0.72±0.00
		MR↓	48.1±1.67	43.0±3.37	36.6±3.14	22.4±3.66	24.4±3.01	19.2±2.72	**18.7±2.53**
Xi'an	ASI	MAE↓	6.33±0.03	5.79±0.12	**5.36±0.03**	5.63±0.01	5.41±0.07	5.38±0.07	5.67±0.02
		RMSE↓	13.1±0.37	11.2±0.10	**10.6±0.01**	10.8±0.12	10.7±0.05	10.8±0.07	10.9±0.06
	TTE	MAE↓	137±6.67	134±5.09	110±4.26	122±4.52	110±3.68	115±15.4	**108±6.82**
		RMSE↓	199±8.23	203±11.9	**169±5.63**	191±3.62	172±8.94	170±13.9	178±8.34
	STS	ACC@3↑	0.71±0.01	0.74±0.04	0.72±0.07	**0.80±0.03**	0.73±0.01	0.75±0.04	0.77±0.02
		MR↓	12.9±1.71	12.1±1.48	12.5±3.62	16.5±4.28	19.0±2.95	**10.8±3.54**	11.9±3.15
Porto	ASI	MAE↓	4.39±0.00	4.40±0.00	4.29±0.00	4.36±0.01	**4.20±0.01**	4.21±0.00	4.35±0.05
		RMSE↓	7.90±0.00	**7.58±0.00**	7.78±0.00	7.93±0.05	7.71±0.01	7.79±0.04	7.79±0.05
	TTE	MAE↓	99.5±1.78	101±1.98	97.2±1.38	98.4±1.81	98.5±2.83	97.5±6.92	**96.8±1.30**
		RMSE↓	150±3.60	147±2.60	143±1.80	147±2.48	**140±2.22**	158±7.04	145±1.03
	STS	ACC@3↑	0.80±0.10	0.82±0.11	0.83±0.03	**0.90±0.06**	0.83±0.04	0.88±0.02	0.87±0.04
		MR↓	14.3±1.91	12.0±1.81	17.9±3.40	12.1±4.03	17.2±1.80	13.7±2.44	**11.4+3.38**
San Francisco	ASI	MAE↓	2.98±0.00	2.51±0.00	2.49±0.01	2.56±0.00	2.48±0.01	2.45±0.00	**2.42±0.01**
		RMSE↓	5.80±0.00	5.38±0.01	5.29±0.01	5.29±0.00	**5.25±0.00**	5.33±0.00	5.37±0.00
	TTE	MAE↓	289±1.02	288±1.01	279±3.21	274±9.01	270±7.61	**268±10.2**	275±9.30
		RMSE↓	1479±4.3	1449±4.4	1499±3.6	**1372±28**	1451±16	1497±38	1446±21
	STS	ACC@3↑	0.84±0.04	0.91±0.00	0.86±0.07	**0.94±0.04**	0.92±0.01	0.90±0.07	0.87±0.05
		MR↓	7.20±3.19	7.00±0.01	8.48±2.05	9.22±3.29	6.55±1.22	6.71±2.01	**6.49±1.63**
Per Avg Rank			6.3	4.9	3.9	3.9	3.2	2.9	**2.8**

1) *Importance of Sequence-based Encoders*: Models integrating token- and sequence-based encoders (*e.g.*, , CTLE, CACSR, Hier) outperform token-based models, indicating the value of trajectory data. Among these, models using Augmented Trajectory Contrastive Learning (ATrCL) and Masked Trajectory Recovery (MTR) tasks (*e.g.*, , CTLE, CACSR) achieve better performance than those focused on Trajectory Prediction (TrajP) tasks (*e.g.*, , Hier). This is likely because ATrCL and MTR capture long-range temporal dependencies across entire trajectories, while TrajP focuses only on consecutive POI dependencies.

2) *Impact of Relation Networks*: For token-based models, those incorporating relation networks (*e.g.*, , Teaser, P2Vec) outperform models without them (*e.g.*, , SkipGram, Tale), highlighting their value. However, for models combining token- and sequence-based encoders, relation networks provide little benefit. This is likely due to the sparsity of POI trajectories, where the number of trajectories between two POIs is often dictated by geographical proximity. As a result, the information provided by relation networks overlaps significantly with that from trajectory data. Once the dependency within the trajectory data has been fully exploited, the relational network contributes limited additional information.

3) *TokRI vs. TRCL Tasks*: Token-based models utilizing the Token Relation Inference (TokRI) task, such as P2Vec, generally outperform those employing Token Relation Contrastive Learning (TRCL), such as SkipGram and Teaser. With sufficient data, supervised learning via TokRI effectively leverages labeled data to generate more discriminative representations. In contrast, the indirect nature of contrastive learning in TRCL offers less direct utilization of labeled information, resulting in comparatively lower model performance.

Table 5: Performance comparison of land parcel MapRL methods on various datasets.

Data	Task	Metric	ZEMob	GMEL	HDGE	MVURE	MGFN	ReMVC	HREP
Beijing	LPC	ACC@1↑	0.43±0.03	0.46±0.03	0.41±0.00	0.48±0.00	0.59±0.02	**0.75±0.02**	0.50±0.01
		F1↑	0.11±0.00	0.61±0.03	0.22±0.01	0.28±0.01	0.65±0.04	**0.71±0.02**	0.38±0.01
	FI	MAE↓	30.9±0.03	27.7±0.01	20.8±2.91	29.6±0.83	30.1±0.02	27.8±0.02	**20.6±0.33**
		RMSE↓	55.3±0.04	59.0±0.04	44.8±2.79	53.7±1.36	54.5±0.02	52.7±0.04	**42.6±0.86**
	MI	MAE↓	0.43±0.03	0.46±0.03	0.34±0.03	**0.27±0.01**	0.30±0.03	0.30±0.01	0.32±0.00
		RMSE↓	0.80±0.02	0.74±0.02	0.70±0.03	0.71±0.02	0.76±0.04	0.69±0.02	**0.66±0.01**
Chengdu	LPC	ACC@1↑	0.36±0.01	0.38±0.04	0.42±0.03	0.48±0.00	0.48±0.08	**0.66±0.03**	0.54±0.01
		F1↑	0.17±0.03	0.16±0.03	0.27±0.06	0.27±0.01	0.35±0.12	**0.59±0.03**	0.46±0.01
	FI	MAE↓	155±0.03	155±0.02	159±2.02	156±3.10	158±5.69	**148±0.02**	156±2.8
		RMSE↓	441±0.02	441±0.04	403±1.12	443±4.28	**337±24.4**	431±0.03	396±9.7
	MI	MAE↓	**3.64±0.03**	4.62±0.04	5.17±0.01	3.97±0.71	5.14±0.02	5.29±0.04	5.47±0.12
		RMSE↓	29.7±0.02	29.7±0.03	28.7±0.15	29.6±0.12	29.3±1.14	29.1±0.06	**28.3±0.07**
Xi'an	LPC	ACC@1↑	0.36±0.03	0.36±0.03	0.44±0.02	0.53±0.01	0.48±0.09	0.57±0.00	**0.59±0.01**
		F1↑	0.19±0.04	0.42±0.03	0.25±0.01	0.26±0.02	0.28±0.07	**0.46±0.03**	0.45±0.02
	FI	MAE↓	134±0.03	133±0.02	124±5.87	127±3.85	**119±9.30**	132±0.04	129±0.80
		RMSE↓	326±0.02	324±0.03	289±26.3	318±4.77	**241±28.4**	325±0.03	297±9.10
	MI	MAE↓	3.83±0.03	3.71±0.04	3.92±0.03	**3.33±0.35**	3.70±0.11	3.91±0.05	4.02±0.04
		RMSE↓	17.7±0.03	17.6±0.03	**16.6±0.14**	17.4±0.00	16.8±1.32	16.8±0.05	16.7±0.20
Porto	LPC	ACC@1↑	0.34±0.04	0.33±0.04	0.42±0.05	0.30±0.01	0.30±0.01	**0.44±0.03**	0.36±0.02
		F1↑	0.21±0.01	0.38±0.03	0.41±0.07	0.27±0.02	0.20±0.03	**0.41±0.04**	0.32±0.03
	FI	MAE↓	528±0.03	525±0.01	501±18.0	402±21.8	477±37.3	527±0.01	**289±12.1**
		RMSE↓	595±0.02	592±0.04	582±14.4	472±21.1	443±36.8	595±0.03	**355±27.3**
	MI	MAE↓	44.9±0.02	41.1±0.03	30.8±4.57	37.8±0.65	29.8±7.26	39.4±0.05	**28.3±5.30**
		RMSE↓	72.3±0.02	56.2±0.04	45.7±4.91	54.1±4.68	45.5±9.79	56.7±0.03	**42.2±8.93**
San Francisco	LPC	ACC@1↑	0.84±0.02	0.64±0.01	0.82±0.01	0.84±0.02	0.83±0.01	**0.86±0.03**	0.84±0.01
		F1↑	0.53±0.03	0.56±0.01	0.50±0.03	0.52±0.05	0.44±0.10	0.47±0.04	**0.57±0.03**
	FI	MAE↓	509±0.04	514±0.01	387±23.1	349±23.7	356±46	445±0.01	**324±25.8**
		RMSE↓	910±0.04	917±0.02	693±35.8	700±44.2	748±80	852±0.02	**624±31**
	MI	MAE↓	**5.88±0.03**	7.40±0.02	6.51±0.51	6.46±0.42	6.40±0.84	6.87±0.04	5.89±0.50
		RMSE↓	23.5±0.01	22.3±0.01	21.5±0.28	20.6±1.21	20.2±2.34	22.7±0.03	**19.1±1.51**
Per Avg Rank			5.67	5.10	4.00	3.73	3.53	3.50	**2.43**

4) *Instability Introduced by Contrastive Learning*. Models that rely on contrastive learning tasks, such as SkipGram, Teaser, and CACSR, exhibit instability due to their sensitivity to dataset partitioning and negative sample selection. In contrast, P2Vec and Tale, which use TokRI—a supervised pretraining task—demonstrate greater stability by explicitly modeling relationships between map entities.

4.2.2 Experimental Results for Road Segment MapRL. Table 4 shows a performance comparison of road segment MapRL models across three downstream tasks: Average Speed Inference (ASI), Travel Time Estimation (TTE), and Similarity Trajectory Search (STS). The following key observations can be drawn from the results:

1) *Full Pipeline Models Perform Best*: Models utilizing the complete token-graph-sequence pipeline, such as JCLRNT and START, achieve the highest performance compared to models with partial pipelines. Additionally, token-based + graph-based and token-based + sequence-based models – such as HRNR, SARN, Toast, and HRoad – outperform purely token-based models like RN2Vec, highlighting the value of integrating trajectory and relation network information. Graph-based and sequence-based encoders capture essential patterns that improve road segment representations.

2) *Multiple Pre-training Tasks Improve Performance*: Models incorporating multiple pre-training tasks, such as Toast, HRoad, START, and JCLRNT, consistently outperform those using a single pre-training task, like HRNR and SARN. Among token-based + graph-based models, HRoad, which integrates both NFI and GAu tasks, surpasses HRNR (GAu only) and SARN (AGCL only), highlighting the benefits of leveraging diverse pre-training tasks. By extracting information from multiple perspectives, these models learn more comprehensive feature representations. Additionally, in full-pipeline models, JCLRNT, which combines a sequence-based task (Augmented Trajectory Contrastive Learning) with a graph-based

task (Augmented Graph Contrastive Learning), outperforms START, which employs two sequence-based tasks (MTR and ATrCL). This suggests that even among multi-task models, greater task diversity further enhances performance.

3) *Fluctuating Results for SARN, Toast, START, and JCLRNT:* These models experience higher performance variability compared to other models, largely due to their dependence on contrastive pre-training techniques such as Token Relation Contrastive Learning (TRCL), Augmented Graph Contrastive Learning (AGCL), and Augmented Trajectory Contrastive Learning (ATrCL). These methods are particularly vulnerable to shifts in training data splits and the inherent randomness in negative sample selection, leading to inconsistent results across multiple runs.

4.2.3 Experimental Results for Land Parcel MapRL. Table 5 presents the performance of land parcel MapRL models on the downstream tasks of Land Parcel Classification (LPC), Flow Inference (FI), and Mobility Inference (MI). The following observations can be made:

1) *Incorporating POI Data Enhances Performance*: Models that utilize POI data, such as MVURE, ReMVC, and HREP (see the "Map Entities" column in Table 1), outperform those that do not. These models treat POI categories within a land parcel as features, enriching the representation of the parcel's functional characteristics. This approach provides valuable contextual information, improving the overall performance of the models.

2) *Limitations of Current Encoder Structures*: Most existing land parcel MapRL models adopt a token-based + graph-based encoder structure, focusing on extracting information from relation networks. However, these models overlook the sequential dependencies within trajectories, limiting their effectiveness. Future research should explore advanced methods to capture trajectory sequence information relevant to land parcels for further improvements.

3) *Fluctuating Results for MGFN*: Although MGFN ranks third overall (with an average rank of 3.53), its performance varies considerably under different random seeds, indicating a degree of instability. This instability arises because MGFN relies on clustering techniques to build the base relation network for representation learning, making it more sensitive to randomness.

4) *Small Datasets Mitigate Instability in Negative Sampling*: ZEMob, GMEL, and ReMVC are among the most stable models across all datasets. Despite employing contrastive learning as pretraining tasks—which tends to be sensitive in other MapRL models—their stability may be attributed to the smaller number of parcel entities. For instance, San Francisco contains 194 parcel entities compared to 27,274 road segment entities. This smaller number of parcel entities likely results in more consistent negative sampling. Additionally, tasks for land parcels are simpler, focusing on static attribute inference rather than complex sequential predictions, which further contributes to stable learning.

4.2.4 Summary for Overall Experiments. The overall experimental results highlight the following key findings across POI, road segment, and land parcel MapRL models.

1) *Importance of Combined Encoders*: Models employing a combination of token-based, graph-based, and sequence-based encoders consistently outperform those relying on single-type encoders. This demonstrates the value of integrating multiple data sources – such as trajectories and relation networks – to capture complex spatial, temporal, and relational patterns.

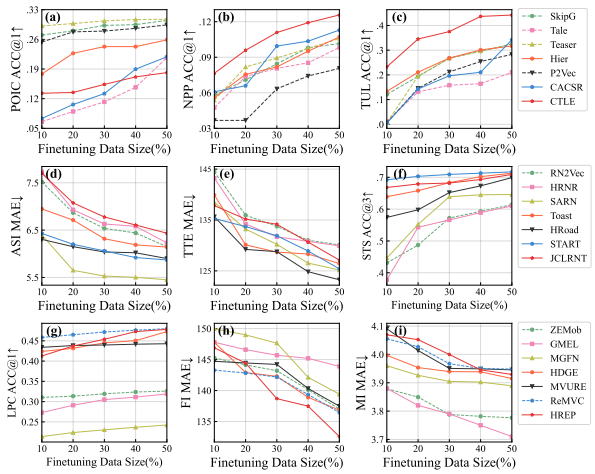


Figure 3: Results from varying fine-tuning data size.

2) *Effectiveness of Diverse Pre-training Tasks*: Models that leverage multiple pre-training tasks usually perform better than those limited to a single task. This suggests that diverse pre-training strategies enhance the model's ability to learn comprehensive feature representations from different perspectives.

3) *Role of Auxiliary Data*: Incorporating trajectory, and relation network data improves model performance across tasks. However, models that fully exploit trajectory dependencies achieve diminishing gains from relation networks, indicating some redundancy between these data sources.

4) Instability Introduced by Contrastive Learning: Contrastive learning relies on the generation of positive and negative sample pairs, which vary between different random seeds. Consequently, models using contrastive learning pretraining tasks are more sensitive to randomness. However, this sensitivity may decrease when working with smaller datasets.

4.3 Scalability Analysis of MapRL Models.

Scalability is a critical factor in MapRL model design, determining their adaptability to varying data sizes and computational constraints. In this section, we analyze scalability from three key perspectives: effectiveness, efficiency, and the trade-offs between model size and inference time. Effectiveness assesses how well models maintain performance as the amount of fine-tuning data varies. Efficiency evaluates computational costs, including parameter count, training time, and inference time, across datasets with different entity scales. Finally, the trade-off analysis explores how model complexity impacts inference speed, offering insights into balancing accuracy and deployment feasibility.

4.3.1 Effectiveness. In this experiment, we adjust the fine-tuning data size from 10% to 50% of the original training set and compare the result of pre-built models in *VecCity*. POI-oriented models are tested on the Tokyo dataset, while road segment- and land parcel-oriented models are evaluated on the Xi'an dataset. Fig. 3 presents the evaluation result: Fig. 3 (a) to (c) show results for POI-oriented models. Fig. 3 (d) to (f) display results for road-segment-oriented

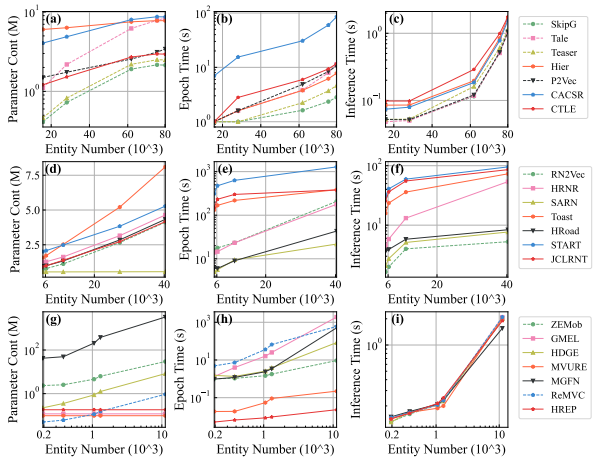


Figure 4: Scalability of MapRL models in model size, training time, and inference time across datasets with varying entity counts. Each point represents a dataset, with lines connecting them in ascending entity count to highlight trends.

models. Fig. 3 (g) to (i) provide results for land-parcel-oriented models. From these figures, we make the following observations:

1) *Performance Scalability.* Token-based models (dashed lines) show limited scalability, with performance gains plateauing around 30% of fine-tuning data. In contrast, graph- and sequence-based models continue improving up to 50%. Notably, methods incorporating contrastive learning techniques—such as CACSR, START, JCLRNT, and HREP—show particularly strong improvements, suggesting that contrastive learning further enhances performance as more data becomes available.

2) *Stability in Attribute Inference Tasks*: MapRL models exhibit stable performance in attribute inference tasks (POIC, ASI, LPC) regardless of training data size. Since these tasks primarily rely on entity attributes learned during pre-training, only a small amount of labeled data is needed for fine-tuning.

3) *Fluctuations in Trajectory-Related and Mobility Inference Tasks*: Models exhibit notable performance fluctuations in trajectory-related tasks (e.g., NPP, TUL, TTE) and mobility inference tasks (e.g., FI, MI). Trajectory-related tasks require sequence-level predictions, whereas most encoders and pre-training tasks generate single-entity embeddings, making these tasks especially sensitive to smaller fine-tuning datasets. By contrast, Similarity Trajectory Search (STS) relies on similarity comparisons rather than prediction, making it more stable. Additionally, Flow Inference (FI) and Mobility Inference (MI) capture human mobility flows that vary greatly over time. This temporal heterogeneity demands more labeled data to accurately model these patterns, leading to further performance variability when data are limited.

Overall, token-based models show weaker performance scalability, with diminishing returns as dataset size increases. In contrast, graph-based and sequence-based models, especially those utilizing contrastive learning methods (e.g., CACSR, START, JCLRNT, HREP), show greater performance improvements as labeled data increases, demonstrating stronger scalability in data-intensive scenarios. Moreover, existing MapRL models exhibit varying sensitivity

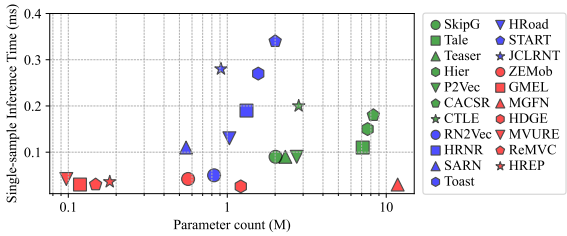


Figure 5: Tradeoff between model parameters and time efficiency on average, across all available datasets.

to the size of labeled data across different tasks. For tasks less influenced by temporal information, such as attribute inference (ASI, POIC, LPC) and Similarity Trajectory Search (STS), models maintain stable performance even with limited training data. However, for tasks with time-dependent labels, such as trajectory-related tasks and mobility inference tasks, larger amounts of labeled data are necessary to effectively capture temporal dynamics.

4.3.2 Efficiency. We evaluate model's efficiency across three key aspects: (1) Parameter count, representing the space complexity of the models; (2) Epoch time, indicating the time cost of training per epoch; and (3) Inference time, reflecting the time required for downstream task evaluation. These measurements are collected from all five datasets used in the main experiments, each with different entity counts. Figure. 4 presents the evaluation results: Fig. 4 (a) to (c) show results for POI-oriented models, Fig. 4 (d) to (f) display results for road segment-oriented models, and Fig. 4 (g) to (i) provide results for land parcel-oriented models. The horizontal axis of all figures represents the number of entities on a 10^3 scale, while the vertical axis represents the parameter count in millions, time cost per epoch in seconds, and total inference time in seconds, respectively. Based on these figures, we have several key findings:

1) *Parameter Count:* Token-based models (dashed lines) generally achieve better scalability in model size, while sequence-based models (e.g., CACSR, Toast, START) often have higher parameter counts, likely due to the increased space complexity of sequence encoders. In contrast, SARN, GMEL, MVURE, and HREP show relatively stable parameter counts as the number of entities increases, demonstrating better scalability in scenarios with large numbers of map entities.

2) *Training Time:* Token-based models (dashed lines) demonstrate better training time scalability in POI-oriented models but perform less efficiently in land parcel-oriented models. This difference is influenced by entity count variations, with POI datasets containing 1×10^4 to 8×10^4 entities, whereas land parcel datasets range from 2×10^2 to 1×10^4. The results suggest that token-based models may have a greater competitive advantage in large-scale datasets. Sequence-based models (e.g., CACSR, Toast, START) generally incur longer training times. Graph-based models employing GAu pretraining tasks (HRNR, GMEL, MGFN) show more-than-linear growth in training time, posing scalability challenges for large datasets, whereas HREP, despite using GAu pretraining, maintains relatively stable training time due to its lower space complexity.

3) *Inference Time:* All models show an increasing trend in inference time. However, the rise is more pronounced in POI- and

Table 6: Available pretraining tasks by entity types.

Task Type	POI	Road Segment	Land Parcel
Token-based	TokRI, TRCL	TokRI, TRCL	TokRI, TRCL, AToCL
Graph-based	–	NFI, GAu,AGCL	NFI, GAu,NCL
Seq-based	TrajP, MTR, ATrCL	TrajP, MTR, ATrCL	–

land parcel-oriented models compared to road segment-oriented models, suggesting that inference time is more sensitive to entity count in these tasks. Sequence-based models (e.g., CTLE, Toast, START, JCLRNT) generally require longer inference times. This can be attributed to their reliance on sequential encoders, which introduce additional computational steps.

4.3.3 Trade-offs. In Section 4, we observe that complex models tend to achieve higher accuracy (*e.g.*, full pipeline models). However, increased model complexity can lead to efficiency challenges, particularly in inference time. The previous efficiency analysis does not directly examine the trade-off between model complexity and inference time. To address this, we evaluate the relationship between parameter count (millions) and single-sample inference time (ms). This experiment uses the same dataset as in Section 4.3.1 on effectiveness: POI-oriented models are evaluated on the Tokyo dataset, while others are tested on the Xi'an dataset.

Fig. 5 presents the results, showing that land-parcel-oriented models maintain relatively stable inference times despite increases in model size, likely due to the lower complexity of their downstream tasks. In contrast, POI and road segment-oriented models exhibit a clear upward trend in inference time as model size grows. Sequence-based models (e.g., CTLE, JCLRNT, Toast, START) have higher model size and inference times due to the integration of sequence encoders. Meanwhile, graph-based models like MVURE, GMEL, ReMVC, and HREP generally maintain lower inference times and smaller model sizes. However, HRNR incurs higher inference costs due to its multi-level GNN encoder.

5 DELVING INTO PRETRAINING TASKS.

In our experimental analysis in Section.4, we conclude that utilizing multiple pretraining tasks can enhance the performance of MapRL models. Moreover, greater task heterogeneity—incorporating token-based, graph-based, and sequence-based pretraining tasks—can further improve model capability. To fully investigate the influence of task heterogeneity on performance, we conduct deep dive experiments to figure out what specific combinations of heterogeneous pretraining tasks can yield the best gains on MapRL.

Experimental Space. In this section, we aim to identify the best pretraining task combinations for MapRL. The candidate combination set is constructed by combining different categories of pretraining tasks. For POI entities, we consider token-based pretraining tasks (TokRI, TRCL) and sequence-based pretraining tasks (TrajP, MTR, ATrCL), forming combinations such as TokRI+TrajP, TRCL+MTR, *etc.* Similarly, for road segments and land parcels, candidate sets are formed by combining token-, graph-, and sequence-based pretraining tasks (if applicable), as shown in Table 6.

The candidate set is constructed based on three rules: (1) selecting pretraining tasks implemented and validated by existing models; (2) using available datasets for testing; and (3) repeating all experiments five times with different random seeds. To ensure experimental feasibility, we further narrow the experimental scope by selecting only one pretraining task from each category for candidate combinations. The candidate set for other entity types follows the same principle.

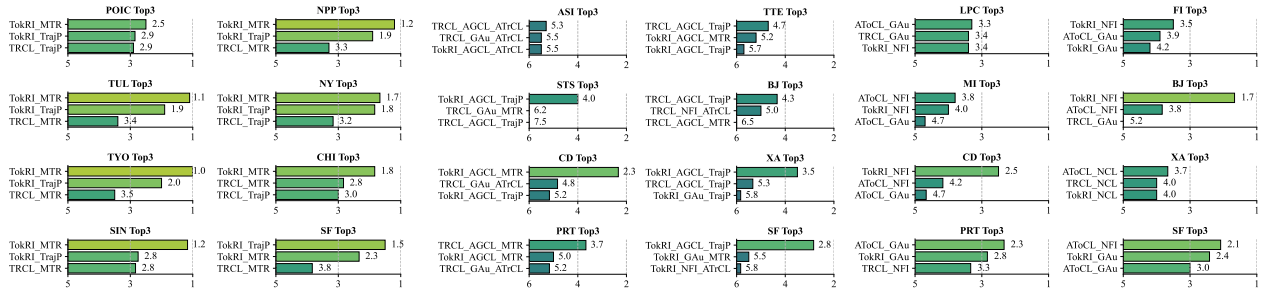
Model Architecture. To eliminate the influence of model architecture, we use the same encoder within each type of MapRL model and include only the necessary encoding components. Specifically, token-based encoders map entity IDs to embeddings. Additionally, the token encoder for POI incorporates spatial features by transforming coordinates into a vector using an MLP, while the token encoder for land parcel integrates semantic features in a similar way. Graph-based encoders use GCNs to process relation networks. For land parcels, the relation network defines edges using POI similarity, spatial distance, and human mobility transitions, following HREP [88]. Sequence-based encoders leverage Transformer layers to encode sequential auxiliary data. The model is configured with the same setting as the main experiment in Section 4. Encoders are arranged in the order: Token → Graph → Sequence.

Experimental Result. To ensure a focused discussion, we highlight the top three pretraining task combinations for each entity type, ranked by their average performance across tasks and datasets. The results are shown in Fig. 6, where each subfigure corresponds to a specific downstream task or dataset. The horizontal axis represents rankings (lower is better), while the vertical axis lists the top three combinations, with lighter colors indicating lower ranks. In Fig. 6(a) for POI entities, the first three subfigures show each model's average ranking in a specific downstream task (POIC, NPP, or TUL) across multiple datasets. We refer to these as *task-oriented rankings*. In contrast, the last five subfigures show each model's average ranking across all tasks within a specific dataset (NY, TYO, CHI, SIN, or SF), which we refer to as *dataset-oriented rankings*. The same ranking logic applies to Fig. 6(b) for road segments and Fig. 6(c) for parcels. Based on the results, we have several key findings:

1) *Optimal Combinations for POI:* TokRI + MTR consistently ranks highest, likely due to its ability to model POI categories and long-range temporal dependencies—two key attributes for POIs. However, in SF, where data sparsity is more pronounced, TokRI + TrajP emerges as a more suitable alternative, as TrajP focuses on local temporal patterns, making it more reliable in sparse data scenarios.

2) *Optimal Combinations for Road Segment:* AGCL outperforms others in all scenarios, as it effectively captures important spatial dependencies in geographic relation (GR) networks, which is critical for modeling road segments. From a dataset-oriented perspective, AGCL + MTR achieves the best results in CD and PRT, where visit frequencies are higher (CD: 1551, PRT: 2430), allowing MTR to learn sequential dependencies from dense trajectory data. In contrast, AGCL + TrajP works better in BJ and SF, where visit frequencies are lower (BJ: 856, SF: 771). This suggests that MTR benefits more from dense data, while TrajP is better suited for sparse data.

3) *Optimal Combinations for Land Parcel:* AToCL achieves the best performance in 5/8 cases, likely because its augmentation technique enhances the model's ability to capture key semantic features while filtering out noise. This is especially important for land parcels, where POI category distributions define functional attributes. GAu and NFI are also well-suited for land parcels, as they better model similarity-driven connectivity in social relation (SR) networks.



(a) Top 3 combinations for POI map entity. (b) Top 3 combinations for segment map entity. (c) Top 3 combinations for parcel map entity.

Figure 6: Top 3 pretraining task combinations.

By integrating these insights, we provide a general guide for selecting pretraining tasks across different entity types:

1) *Token-based*: Encoding semantic features is a key capability of token-based encoders. TokRI is well-suited for structured categorical attributes, while AToCL's augmentation-based contrastive learning better handles high-dimensional and redundant semantic features, making it particularly effective for land parcels.

2) *Graph-based*: AGCL is well-suited for geographic relation networks, as it effectively captures key spatial dependencies. In contrast, GAu and NFI perform better in social relation networks due to their ability to model similarity-driven connectivity.

3) *Sequence-based*: MTR excels at capturing long-range sequential dependencies, making it the best choice for sequence-based tasks, but it requires dense data to be effective. In contrast, TrajP is more reliable in sparse data scenarios.

6 RELATED WORK

Niche Tasks for MapRL. Raster and vector data models are the two primary formats for electronic maps. The raster model structures spatial information through grids, which are commonly used in remote sensing and environmental analysis. In contrast, the vector model represents map entities as points, polylines, and polygons, leveraging graph structures to represent relations between entities. These structural differences lead to different modeling approaches: CNNs are usually used for raster-based learning, while GNNs better capture spatial relations in vector data. As a result, raster and vector models have distinct benchmarks. Due to its high precision and accuracy, the vector data model serves as the fundamental representation for electronic maps.

In parallel, various downstream tasks in MapRL have been widely explored, including Next POI Prediction (NPP), Travel Time Estimation (TTE), and Mobility Inference (MI). These tasks play a key role in location-based services and spatiotemporal data mining. For NPP, models like DeepMove[20] employ hierarchical RNNs with attention mechanisms to capture sequential patterns in user trajectories. TTE methods such as DeepTTE [62] integrate LSTM networks with dynamic graph convolutional layers, fusing traffic state and weather data for accurate time estimation. For MI, models like ODMP [68] adopt a full-pipeline approach, integrating sequence and graph encoders to effectively capture demand-supply dynamics in ride-hailing scenarios.

Reviews for MapRL. Pervious reviews [2, 30, 66] discuss the applications of deep learning in urban data mining, providing a detailed overview of data, tasks, and deep learning models, particularly those that integrate sequence and graph encoders. However, their focus is on end-to-end models, while our work concentrates on pre-trained representation learning methods. Other studies [9, 86] focus on data mining methods for trajectory data but do not cover pre-trained representation learning. Survey [10] introduces a type-based taxonomy for MapRL where models are systematically reviewed based on the types of map entities they target. In contrast, our work does not classify methods by entity type but instead focuses on summarizing general techniques applicable to all map entities, *i.e.,* a method-based taxonomy. This broader perspective helps uncover common design principles underlying MapRL methods.

Benchmark for MapRL. The growing interest in MapRL has increased the demand for open benchmarks to analyze baseline models. Open-source benchmarking is valuable not only for advancing research but also for enabling users to evaluate models and apply them to open datasets. While significant progress has been made in benchmarking for multivariate time series prediction [51, 64, 67], computer vision [14], and neural language processing [50], benchmarking in the MapRL field remains largely unexplored. To our knowledge, *VecCity* is the first MapRL benchmark for comparative experiments and model development.

7 CONCLUSION

To overcome the challenges of fragmentation and the lack of standardized benchmarks in MapRL, we introduced a novel taxonomy organized by functional modules rather than entity types. Building on this taxonomy, we developed *VecCity*, a modular library offering interfaces for encoding, pre-training, fine-tuning, and evaluation. Using *VecCity*, we established the first standardized benchmarks by reproducing 21 mainstream models and integrating datasets from nine cities. Moreover, we also conduct an in-depth evaluation of mainstream MapRL models. Our comprehensive experiments highlight the impact of pre-training tasks and encoder architectures, demonstrating the advantages of combining multiple components. VecCity provides a unified framework that promotes reusability, streamlines experimentation, and advances research in MapRL. Although *VecCity* currently supports most mainstream MapRL models, we plan to expand it further to more pre-trained spatiotemporal data representation learning algorithms.

REFERENCES

[1] 2022. DiDi GAIA Open Dataset. https://outreach.didichuxing.com/research/opendata/

[2] Gowtham Atluri, Anuj Karpatne, and Vipin Kumar. 2018. Spatio-Temporal Data Mining: A Survey of Problems and Methods. *ACM Computing Surveys, CSUR* 51, 4 (2018), 1–41.

[3] Junxiang Bing, Meng Chen, Min Yang, Weiming Huang, Yongshun Gong, and Liqiang Nie. 2023. Pre-Trained Semantic Embeddings for POI Categories Based on Multiple Contexts. *IEEE Transactions on Knowledge and Data Engineering* 35, 9 (Sep 2023), 8893–8904. https://doi.org/10.1109/tkde.2022.3218851

[4] Howard Butler, Martin Daly, Allan Doyle, Sean Gillies, Stefan Hagen, and Tim Schaub. 2016. *The geojson format.* Technical Report.

[5] Buru Chang, Yonggyu Park, Donghyeon Park, Seongsoon Kim, and Jaewoo Kang. 2018. Content-Aware Hierarchical Point-of-Interest Embedding Model for Successive POI Recommendation. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI.*

[6] Yanchuan Chang, Egemen Tanin, Xin Cao, and Jianzhong Qi. 2023. Spatial Structure-Aware Road Network Embedding via Graph Contrastive Learning. In *Proceedings 26th International Conference on Extending Database Technology, EDBT.* 144–156.

[7] Meng Chen, Zechen Li, Weiming Huang, Yongshun Gong, and Yilong Yin. 2024. Profiling Urban Streets: A Semi-Supervised Prediction Model Based on Street View Imagery and Spatial Topology. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, SIGKDD*, Vol. 30. 319–328.

[8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning, ICML.* 1597–1607.

[9] Wei Chen, Yuxuan Liang, Yuanshao Zhu, Yanchuan Chang, Kang Luo, Haomin Wen, Lei Li, Yanwei Yu, Qingsong Wen, Chao Chen, et al. 2024. Deep Learning for Trajectory Data Management and Mining: A Survey and Beyond. *arXiv preprint arXiv:2403.14151* (2024).

[10] Yile Chen, Weiming Huang, Kaiqi Zhao, Yue Jiang, and Gao Cong. 2024. Self-supervised Learning for Geospatial AI: A Survey. *arXiv preprint arXiv:2408.12133* (2024).

[11] Yile Chen, Xiucheng Li, Gao Cong, Zhifeng Bao, and Cheng Long. 2024. Semantic-Enhanced Representation Learning for Road Networks with Temporal Dynamics. *arXiv preprint arXiv:2403.11495* (2024).

[12] Yile Chen, Xiucheng Li, Gao Cong, Zhifeng Bao, Cheng Long, Yiding Liu, Arun Kumar Chandran, and Richard Ellison. 2021. Robust road network representation learning: When traffic patterns meet traveling semantics. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management, CIKM.* 211–220.

[13] Kyunghyun Cho. 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078* (2014).

[14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A Large-Scale Hierarchical Image Database. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR.* 248–255.

[15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT.* 4171–4186.

[16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations, ICLR.*

[17] Jiadi Du, Yunchao Zhang, Pengyang Wang, Jennifer Leopold, and Yanjie Fu. 2019. Beyond Geo-First Law: Learning Spatial Representations via Integrated Autocorrelations and Complementarity. In *IEEE International Conference on Data Mining, ICDM*, Vol. 5. 160–169.

[18] Environmental Systems Research Institute. 1998. *ESRI Shapefile Technical Description.* Technical Report. ESRI, Redlands, California. https://www.esri.com/library/whitepapers/pdfs/shapefile.pdf An ESRI White Paper.

[19] Ziqi Fang, Jichao Zhu, Jun Gao, Haiping Huang, and Chengqi Zhang. 2020. Trajectory Data Augmentation with Adversarial Learning. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, SIGKDD.* 2269–2277.

[20] Jie Feng, Yong Li, Chao Zhang, Funing Sun, Fanchao Meng, Ang Guo, and Depeng Jin. 2018. Deepmove: Predicting Human Mobility with Attentional Recurrent Networks. In *Proceedings of the 2018 World Wide Web Conference, WWW.* 1459–1468.

[21] Shanshan Feng, Gao Cong, Bo An, and Yeow Meng Chee. 2017. POI2Vec: Geographical Latent Representation for Predicting Future Visitors. In *Proceedings of the AAAI Conference on Artificial Intelligence, AAAI*, Vol. 31.

[22] Letian Gong, Youfang Lin, Shengnan Guo, Yan Lin, Tianyi Wang, Erwen Zheng, Zeyu Zhou, and Huaiyu Wan. 2023. Contrastive Pre-training with Adversarial Perturbations for Check-in Sequence Representation Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence, AAAI*, Vol. 37. 4276–4283.

[23] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. 2020. Array programming with NumPy. *Nature* 585, 7825 (2020), 357–362.

[24] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2020. Momentum Contrast for Unsupervised Visual Representation Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR.* 9729–9738.

[25] Weiming Huang, Lizhen Cui, Meng Chen, Daokun Zhang, and Yao Yao. 2022. Estimating urban functional distributions with semantics preserved POI embedding. *International Journal of Geographical Information Science* (2022). https://doi.org/10.1080/13658816.2022.2040510

[26] Weiming Huang, Daokun Zhang, Gengchen Mai, Xu Guo, and Lizhen Cui. 2023. Learning Urban Region Representations with POIs and Hierarchical Graph Infomax. *ISPRS Journal of Photogrammetry and Remote Sensing* 196 (2023), 134–145.

[27] Tobias Skovgaard Jepsen, Christian S Jensen, Thomas Dyhre Nielsen, and Kristian Torp. 2018. On Network Embedding for Machine Learning on Road Networks: A Case Study on the Danish Road Network. In *2018 IEEE International Conference on Big Data (Big Data).* IEEE, 3422–3431.

[28] Jiawei Jiang, Dayan Pan, Houxing Ren, Xiaohan Jiang, Chao Li, and Jingyuan Wang. 2023. Self-supervised trajectory representation learning with temporal regularities and travel semantics. In *IEEE 39th International Conference on Data Engineering, ICDE.* 843–855.

[29] Yizhu Jiao, Yun Xiong, Jiawei Zhang, Yao Zhang, Tianqi Zhang, and Yangyong Zhu. 2020. Sub-Graph Contrast for Scalable Self-Supervised Graph Representation Learning. In *IEEE International Conference on Data Mining, ICDM.*

[30] Guangyin Jin, Yuxuan Liang, Yuchen Fang, Zezhi Shao, Jincai Huang, Junbo Zhang, and Yu Zheng. 2023. Spatio-Temporal Graph Neural Networks for Predictive Learning in Urban Computing: A Survey. *IEEE Transactions on Knowledge and Data Engineering, TKDE* (2023).

[31] Namwoo Kim and Yoonjin Yoon. 2022. Effective urban region representation learning using heterogeneous urban graph attention network (HUGAT). *arXiv preprint arXiv:2202.09021* (2022).

[32] Yi Li, Weiming Huang, Gao Cong, Hao Wang, and Zheng Wang. 2023. Urban Region Representation Learning with OpenStreetMap Building Footprints. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, SIGKDD.* 1363–1373.

[33] Zechen Li, Weiming Huang, Kai Zhao, Min Yang, Yongshun Gong, and Meng Chen. 2024. Urban Region Embedding via Multi-View Contrastive Prediction. *Proceedings of the AAAI Conference on Artificial Intelligence, AAAI* 38, 8 (Mar 2024), 8724–8732.

[34] Zhonghang Li, Long Xia, Lei Shi, Yong Xu, Dawei Yin, and Chao Huang. 2024. OpenCity: Open Spatio-Temporal Foundation Models for Traffic Prediction. arXiv:2408.10269

[35] Yan Lin, Huaiyu Wan, Shengnan Guo, and Youfang Lin. 2021. Pre-training Context and Time Aware Location Embeddings from Spatial-Temporal Trajectories for User Next Location Prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence, AAAI*, Vol. 35. 4241–4248.

[36] Xin Liu, Yong Liu, and Yuanyuan Liu. 2016. Exploring the Context of Locations for Personalized Location Recommendations. *International Joint Conference on Artificial Intelligence, IJCAI* (Jul 2016).

[37] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019).

[38] Zhicheng Liu, Fabio Miranda, Weiting Xiong, Junyan Yang, Qiao Wang, and Claudio Silva. 2020. Learning Geo-Contextual Embeddings for Commuting Flow Prediction. *Proceedings of the AAAI Conference on Artificial Intelligence, AAAI* (Jun 2020), 808–816. http://dx.doi.org/10.1609/aaai.v34i01.5425

[39] Yan Luo, Fu-lai Chung, and Kai Chen. 2022. Urban Region Profiling via Multi-Graph Representation Learning. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management, CIKM.*

[40] Zhipeng Ma, Zheyan Tu, Xinhai Chen, Yan Zhang, Deguo Xia, Guyue Zhou, Yilun Chen, Yu Zheng, and Jiangtao Gong. 2024. More Than Routing: Joint GPS and Route Modeling for Refine Trajectory Representation Learning. (2024), 3064–3075.

[41] Gengchen Mai, Krzysztof Janowicz, Yingjie Hu, Song Gao, Bo Yan, Rui Zhu, Ling Cai, and Ni Lao. 2022. A Review of Location Encoding for GeoAI: Methods and Applications. *International Journal of Geographical Information Science, IJGIS* 36, 4 (2022), 639–673.

[42] Zhenyu Mao, Ziyue Li, Dedong Li, Lei Bai, and Rui Zhao. 2022. Jointly Contrastive Representation Learning on Road Network and Trajectory. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management, CIKM.*

[43] Wendy Kan Meghan O'Connell, moreiraMatias. 2015. ECML/PKDD 15: Taxi Trajectory Prediction (I). https://kaggle.com/competitions/pkdd-15-predict-taxi-service-trajectory-i

[44] Tomas Mikolov, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. In *International Conference on Learning Representations, ICLR*.

[45] Yansong Ning, Hao Liu, Hao Wang, Zhenyu Zeng, and Hui Xiong. 2024. UUKG: Unified Urban Knowledge Graph Dataset for Urban Spatiotemporal Prediction. *Advances in Neural Information Processing Systems, NeurIPS* 36 (2024).

[46] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).

[47] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP*. 1532–1543.

[48] Matthew E Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. Deep Contextualized Word Representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT*. 2227–2237.

[49] Michal Piorkowski and Matthias Grossglauser. 2009. Dataset of Mobility Traces of Taxi Cabs in San Francisco. https://crawdad.org/epfl/mobility/20090224/

[50] Percy Liang Pranav Rajpurkar, Robin Jia. 2018. Know What You Don't Know: Unanswerable Questions for SQuAD. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, EMNLP*. 784–789.

[51] Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S Jensen, Zhenli Sheng, and Bin Yang. 2024. TFB: Towards Comprehensive and Fair Benchmarking of Time Series Forecasting Methods. *Proceedings of the International Conference on Very Large Databases, VLDB* 17 (2024), 2363 – 2377.

[52] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving Language Understanding by Generative Pre-Training. *OpenAI Blog* (2018).

[53] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the Limits of Transfer Learning with A Unified Text-to-Text Transformer. *Journal of Machine Learning Research, JMLR* 21, 140 (2020), 1–67.

[54] Hasim Sak, Andrew W. Senior, and Françoise Beaufays. 2014. Long short-term memory recurrent neural network architectures for large scale acoustic modeling. In *15th Annual Conference of the International Speech Communication Association, INTERSPEECH*. 338–342.

[55] Stefan Schestakov, Paul Heinemeyer, and Elena Demidova. 2023. Road Network Representation Learning with Vehicle Trajectories. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining, PAKDD*. 57–69.

[56] Toru Shimizu, Takahiro Yabe, and Kota Tsubouchi. 2020. Enabling Finer Grained Place Embeddings Using Spatial Hierarchy from Human Mobility Trajectories. In *Proceedings of the 28th International Conference on Advances in Geographic Information Systems, SIGSPATIAL*. 187–190.

[57] Junho Song, Jiwon Son, Dong-hyuk Seo, Kyungsik Han, Namhyuk Kim, and Sang-Wook Kim. 2022. ST-GAT: A Spatio-Temporal Graph Attention Network for Accurate Traffic Speed Prediction. In *Proceedings of the 31st ACM international conference on information & knowledge management, CIKM*. 4500–4504.

[58] Fengze Sun, Jianzhong Qi, Yanchuan Chang, Xiaoliang Fan, Shanika Karunasekera, and Egemen Tanin. 2024. Urban Region Tepresentation Learning with Attentive Fusion. In *2024 IEEE 40th International Conference on Data Engineering, ICDE*. 4409–4421.

[59] PostGIS Development Team. 2023. PostGIS: Spatial and Temporal SQL for PostgreSQL. https://postgis.net

[60] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Advances in neural information processing systems*. 5998–6008.

[61] Huaiyu Wan, Yan Lin, Shengnan Guo, and Youfang Lin. 2021. Pre-training Time-Aware Location Embeddings from Spatial-Temporal Trajectories. *IEEE Transactions on Knowledge and Data Engineering, TKDE* 34, 11 (2021), 5510–5523.

[62] Dong Wang, Junbo Zhang, Wei Cao, Jian Li, and Yu Zheng. 2018. When will you arrive? Estimating travel time based on deep neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.

[63] Hongjian Wang and Zhenhui Li. 2017. Region Representation Learning via Mobility Flow. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, CIKM*.

[64] Jingyuan Wang, Jiawei Jiang, Wenjun Jiang, Chao Li, and Wayne Xin Zhao. 2021. Libcity: An Open Library for Traffic Prediction. In *Proceedings of the 29th International Conference on Advances in Geographic Information Systems, SIGSPATIAL*. 145–148.

[65] Meng-Xiang Wang, Wang-Chien Lee, Tao-Yang Fu, and Ge Yu. 2020. On Representation Learning for Road Networks. *ACM Transactions on Intelligent Systems and Technology, TIST* 12, 1 (2020), 1–27.

[66] Senzhang Wang, Jiannong Cao, and Philip S. Yu. 2022. Deep Learning for Spatio-Temporal Data Mining: A Survey. *IEEE Transactions on Knowledge and Data Engineering, TKDE* 34, 8 (2022), 3681–3700.

[67] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Yong Liu, Mingsheng Long, and Jianmin Wang. 2024. Deep Time Series Models: A Comprehensive Survey and Benchmark. (2024).

[68] Yuandong Wang, Hongzhi Yin, Hongxu Chen, Tianyu Wo, Jie Xu, and Kai Zheng. 2019. Origin-destination matrix prediction via graph convolution: a new perspective of passenger demand modeling. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 1227–1235.

[69] Ning Wu, Xin Wayne Zhao, Jingyuan Wang, and Dayan Pan. 2020. Learning Effective Road Network Representation with Hierarchical Graph Neural Networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining, SIGKDD*. 6–14.

[70] Shangbin Wu, Xu Yan, Xiaoliang Fan, Shirui Pan, Shichao Zhu, Chuanpan Zheng, Ming Cheng, and Cheng Wang. 2022. Multi-Graph Fusion Networks for Urban Region Embedding. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI*. 2291–2297.

[71] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. 2019. Graph Wavenet for Deep Spatial-Temporal Graph Modeling. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence, IJCAI*. 1907–1913.

[72] Bo Yan, Krzysztof Janowicz, Gengchen Mai, and Song Gao. 2017. From ITDL to Place2Vec: Reasoning About Place Type Similarity and Relatedness by Learning Embeddings From Augmented Spatial Contexts. In *Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. https://doi.org/10.1145/3139958.3140054

[73] Can Yang and Gyozo Gidofalvi. 2018. Fast Map Matching: An Algorithm Integrating Hidden Markov Model with Precomputation. *International Journal of Geographical Information Science, SIGSPATIAL* 32, 3 (2018), 547–570.

[74] Dingqi Yang, Daqing Zhang, and Bingqing Qu. 2016. Participatory cultural mapping based on collective behavior data in location-based social networks. *ACM Transactions on Intelligent Systems and Technology, TIST* 7, 3 (2016), 1–23.

[75] Yao Yao, Qia Zhu, Zijin Guo, Weiming Huang, Yatao Zhang, Xiaoqin Yan, Anning Dong, Zhangwei Jiang, Hong Liu, and Qingfeng Guan. 2023. Unsupervised land-use change detection using multi-temporal POI embedding. *International Journal of Geographical Information Science* 37, 11 (Nov 2023), 2392–2415. https://doi.org/10.1080/13658816.2023.2257262

[76] Zijun Yao, Yanjie Fu, Bin Liu, Wangsu Hu, and Hui Xiong. 2018. Representing Urban Functions through Zone Embedding with Human Mobility Patterns.. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI*.

[77] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2018. Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence, IJCAI*. 3634–3640.

[78] Chenhan Zhang, JQ James, and Yi Liu. 2019. Spatial-Temporal Graph Attention Networks: A Deep Learning Approach for Traffic Forecasting. *IEEE Access* 7 (2019), 166246–166256.

[79] Liang Zhang and Cheng Long. 2023. Road Network Representation Learning: A Dual Graph-based Approach. *ACM Transactions on Knowledge Discovery from Data, TKDD* 17, 9 (2023), 1–25.

[80] Liang Zhang, Cheng Long, and Gao Cong. 2022. Region Embedding with Intra and Inter-view Contrastive Learning. *IEEE Transactions on Knowledge and Data Engineering, TKDE* 35, 9 (2022), 9031–9036.

[81] Mingyang Zhang, Tong Li, Yong Li, and Pan Hui. 2020. Multi-View Joint Graph Representation Learning for Urban Region Embedding. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI*.

[82] Weijia Zhang, Jindong Han, Zhao Xu, Hang Ni, Hao Liu, and Hui Xiong. 2024. Urban Foundation Models: A Survey. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, SIGKDD*. 6633–6643.

[83] Yunchao Zhang, Yanjie Fu, Pengyang Wang, Xiaolin Li, and Yu Zheng. 2019. Unifying Inter-region Autocorrelation and Intra-region Structures for Spatial Embedding via Collective Adversarial Learning. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, SIGKDD*.

[84] Jingkang Zhao, Hao Wang, Xiaolong Ma, and Jun Gao. 2019. Augmentation by Perturbation for Robust Trajectory Prediction. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*. AAAI Press, 5026–5032.

[85] Shenglin Zhao, Tong Zhao, Irwin King, and Michael R Lyu. 2017. Geo-Teaser: Geo-Temporal Sequential Embedding Rank for Point-of-Interest Recommendation. In *Proceedings of the 26th International Conference on World Wide Web Companion, WWW*. 153–162.

[86] Yu Zheng. 2015. Trajectory Data Mining: An Overview. *ACM Transactions on Intelligent Systems and Technology, TIST* 6, 3 (2015), 29:1–29:41.

[87] Haicang Zhou, Weiming Huang, Yile Chen, Gao Cong, and Soon Ong. 2024. Road Network Representation Learning with the Third Law of Geography. *arXiv preprint arXiv:2406.04038* (Jan 2024).

[88] Silin Zhou, Dan He, Lisi Chen, Shuo Shang, and Peng Han. 2023. Heterogeneous Region Embedding with Prompt Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence, AAAI*, Vol. 37. 4981–4989.